PEKEMEN

Videojuego de combate por consola en Java

Proyecto	Proyecto individual incremental · Primera evaluación · Entrega 1
Alumno	Jorge García González
Módulo / grupo	1.º DAW
Curso	2026/2027
Repositorio GitHub	https://github.com/deniozaque/trabajo-de-programacion
Versión	v1.0-primera-evaluacion

Este proyecto ha sido totalmente pensado, elaborado y programado por el autor Jorge, el único apoyo en la inteligencia artificial, internet o personas conocidas ha sido en la redacción de este FDP y en el Readme.

1. Descripción del videojuego

1.1. Temática y objetivo

Pekemen es un videojuego de combate por turnos que se juega por consola. El jugador elige una criatura inicial (los *pekemens*) de uno de los tres elementos del juego (agua, planta o fuego) y la enfrenta a la máquina. Cada criatura tiene vida, un daño máximo y un tipo; además, los tipos se relacionan entre sí con ventajas y desventajas.

El programa ofrece dos modos de juego:

- **Combate singular:** una única batalla entre la criatura elegida y un rival aleatorio. Termina cuando uno de los dos pierde.
- **Modo historia:** tras ganar el primer combate, el jugador decide entre luchar contra un *líder* de gimnasio o buscar pekemens salvajes para *matarlos* (y ganar experiencia) o *capturarlos* (y ampliar su equipo). El objetivo es derrotar a los tres líderes (agua, fuego y planta) y al rival final, que usa un equipo de seis criaturas.

1.2. Reglas básicas

- **Ventaja de tipos:** fuego gana a planta, planta gana a agua y agua gana a fuego. El atacante con ventaja multiplica su daño por 1,25; con desventaja, por 0,75. Entre tipos iguales, o si interviene el tipo *normal*, el multiplicador es 1,0.
- **Daño:** cada ataque calcula una potencia aleatoria entre 10 y (daño máximo + 1) × 10 y la multiplica por la ventaja. Hay un 10 % de probabilidad de golpe crítico (multiplicador 1,1).
- **Equipo:** el jugador puede tener hasta 6 pekemens. Si el pekemen activo cae, debe sacar otro con vida; si no le queda ninguno, pierde el combate.
- **Poción:** cada bando dispone de una sola poción por combate que recupera 20 puntos de vida.
- **Experiencia:** al derrotar a un salvaje, todo el equipo recibe su experiencia; al acumular 100 puntos, el pekemen sube de nivel.

1.3. Acciones del jugador en combate

En cada turno el jugador elige una de las siguientes acciones. Todas modifican el estado de la partida:

N.º	Acción	Efecto sobre el estado de la partida
1	Atacar	Resta vida al rival (potencia aleatoria × ventaja de tipo); 10 % de crítico.
2	Curarte	Suma 20 de vida al jugador. Solo una vez por combate; si no quedan pociones lo avisa.
3	Explotar	El rival queda a 0 de vida (victoria inmediata), pero el jugador pierde la mitad de su vida.
4	Atacar fuerte	40 % de acierto; si acierta, hace el doble de daño. Si falla, informa de ello.
5	Protegerte	El rival no realiza su ataque normal en ese turno.
6	Efecto especial	Según el tipo: fuego quema (−15 de vida por turno), planta envenena (−25 por turno), agua ahoga (el rival cae tras 3 turnos). Solo funciona contra un tipo distinto.
7	Cambiar pekemen	Recorre el equipo, ofrece los pekemens con vida y sustituye al activo conservando su vida actual.

Además de estas acciones, el menú del mundo (modo historia) permite **luchar contra el líder**,

buscar pekemens salvajes y, dentro de este último, **matar** o **capturar**. El menú principal permite elegir modo de juego o **salir**.

1.4. Desarrollo de una partida

- Se muestra el menú principal (modo historia, combate singular o salir) y se repite hasta que la opción sea válida.
- El jugador elige su pekemen inicial (Peplet, Terteg o Chemchemp). La máquina elige otro de los tres al azar.
- Comienza el combate por turnos. En cada turno se muestra el tipo de ambos, la vida de ambos con una barra de almohadillas (una por cada 10 puntos de vida) y el menú de acciones. Tras la acción del jugador actúan los estados alterados y, después, la máquina.
- Cuando cae un pekemen, el combate continúa con el siguiente disponible de ese bando. Cuando un bando se queda sin criaturas, el combate termina.
- En combate singular se vuelve al menú principal. En modo historia se pasa al menú del mundo y la aventura sigue hasta derrotar al rival final o perder un combate.

1.5. Condiciones de victoria y derrota

- **Victoria en un combate:** todos los pekemens del rival llegan a 0 de vida. El programa muestra *Has ganado*.
- **Derrota:** todos los pekemens del jugador llegan a 0 de vida. El programa muestra *Has perdido el combate* y vuelve al menú principal.
- **Victoria del modo historia:** haber derrotado a los tres líderes y al rival final, con lo que termina la aventura.

2. Funcionalidades implementadas

Todo lo que se enumera a continuación está implementado en el archivo `App.java` y ha sido comprobado ejecutando el programa (ver el apartado 4).

Funcionalidad	Estado
Menú principal repetible con validación de rango (1–3) y salida controlada.	Implementada
Elección del pekemen inicial con validación; rival inicial aleatorio.	Implementada
Combate por turnos con siete acciones y barra de vida en pantalla.	Implementada
Sistema de ventajas por tipo (agua / fuego / planta / normal) y golpes críticos.	Implementada
Estados alterados: quemadura, veneno y ahogamiento.	Implementada
Inteligencia artificial sencilla del rival (curarse, crítico, ataque desesperado, ataque normal).	Implementada
Equipo de hasta 6 pekemens, cambio voluntario y cambio forzoso al caer el activo.	Implementada
Modo historia: tres líderes con 3 pekemens cada uno y rival final con 6.	Implementada
Pekemens salvajes: matar (experiencia y nivel) o capturar (probabilidad de éxito).	Implementada
Condiciones de victoria y derrota, y regreso al menú principal.	Implementada

2.1. Diferencias respecto a la propuesta inicial

Ha habido muchos cambios desde la idea principal ha habido muchos cambios:
Han pasado de usar tipos a pekemens, el torneo ha pasado a ser el modo historia y he añadido más opciones.

Catálogo de pekemens incluido en el juego (datos de la matriz `pekemens`):

N.º	Nombre	Tipo	Vida	Daño máx.	Exp. que da
1	Peplet	agua	40	2	40
2	Prenplet	agua	70	4	60
3	Enpeleen	agua	100	6	75
4	Terteg	planta	40	2	40
5	Tercel	planta	70	4	60
6	Terterra	planta	100	6	75
7	Chemchemp	fuego	40	2	40
8	Menferme	fuego	70	4	60
9	Enfermet	fuego	100	6	75
10	Bedef	normal	30	2	20
11	Shex	normal	30	2	30
12	Lexe	normal	65	4	50
13	Lexrrey	normal	85	6	60
14	Tepexes	agua	35	3	35
15	Mereene	agua	70	5	50
16	Slegme	fuego	40	4	40
17	Megcerge	fuego	75	5	65
18	Cecnee	planta	45	5	45
19	Cecterne	planta	100	6	55

3. Diseño e implementación

3.1. Estructura del programa

El proyecto consta de una única clase `App` con el método `main`. La cabecera es un comentario con autor, fecha y descripción; después van los `import` (`BufferedReader`, `InputStreamReader` y `Random`), y dentro de `main` el código se organiza en bloques: constantes, declaración de variables y matrices, inicialización, bucle principal del menú, elección de pekemen inicial, bucle de combate, bucle de turnos y menú del mundo.

3.2. Variables y constantes

Las **constantes** se declaran con `final` y en mayúsculas:

```
final double CRITICO = 1.1;      // multiplicador del golpe crítico
final String FUEGO = "fuego";    // nombres de los tipos
final String AGUA = "agua";
final String PLANTA = "planta";
```

Las **variables** principales, agrupadas por función y tipo elegido:

Variable	Tipo	Para qué sirve
vida_jugador, vida_rival	double	Vida actual de cada bando; admite decimales porque el daño se multiplica por 1,25, 0,75 o 1,1.
potencia, ventaja	double	Daño base calculado en cada ataque y multiplicador de tipo.
fuerza_maxima_jugador / _rival	int	Daño máximo del pekemen activo; limita la potencia aleatoria.
eleccion, eleccion_pekemen, eleccion_salvaje	int	Opciones tecleadas en los distintos menús.
posicion, posicion_rival	int	Índice del pekemen activo dentro de cada equipo (array).
contador_ahogar	int	Turnos transcurridos del efecto «ahogar» (9 significa inactivo).
tipo_jugador, tipo_rival, inicial_rival	String	Tipo de cada pekemen activo y nombre del rival en pantalla.
historia, lider, huir, salir, perder	Boolean	Indicadores de modo de juego y de fin de menú, de combate o de partida.
pocion_jugador, pocion_rival	Boolean	Si cada bando conserva su poción.
quemar, envenenar	Boolean	Estados alterados activos sobre el rival.
save (BufferedReader), ram (Random)	objetos	Lectura de teclado y generación de números aleatorios.

3.3. Operadores y conversiones de tipo

Se usan **operadores aritméticos** para calcular el daño (`vida_rival -= potencia * ventaja * CRITICO`), la barra de vida (`(int) vida_jugador / 10`) y la experiencia; **relacionales** para comparar vida, tipos y opciones (`<=`, `>`, `==`, `!=`); y **lógicos** (`&&`, `||`, `!`) para combinar condiciones, por ejemplo la ventaja de tipo o las decisiones del rival.

Como los datos de cada pekemen se guardan en un array de `String`, aparecen muchas **conversiones explícitas**: `Double.parseDouble` y `Integer.parseInt` para leer vida, daño o experiencia como números, `String.valueOf` y la concatenación con `""` para volver a guardarlos como texto, y el *cast* `(int)` para truncar la vida al dibujar la barra. Como conversión **implícita** está la promoción de `int` a `double` al operar con `ventaja`. La entrada del teclado llega como texto y se convierte con `Integer.parseInt(save.readLine())`.

3.4. Estructuras de control

Estructura	Dónde se usa y para qué
while / do-while	Bucle principal (`while (huir == false)`) que mantiene el juego activo; `do-while` para repetir cada menú hasta recibir una opción válida; bucle de turnos (`while (perder == false)`) y bucle del modo historia (`while (historia == true)`).
for	Recorre los equipos (arrays): comprobar si quedan pekemens vivos, buscar un hueco libre, listar candidatos al cambio, repartir experiencia.
if / else if / else	Reglas del juego: ventaja de tipos, crítico, decisiones de la máquina, victoria y derrota.
switch	Selección de opciones de los menús (principal, pekemen inicial, acciones de combate, salvaje: matar o capturar).
break	Sentencia de salto: sale del `switch` tras cada caso, del bucle de menú al elegir salir y de los recorridos con `for` cuando ya se ha encontrado lo buscado (por ejemplo, el primer hueco libre del equipo).

3.5. Entrada y salida por consola

La entrada se lee con `BufferedReader`. Cada menú muestra las opciones numeradas y pide «Eleccion:». La salida usa `System.out.printf` con formato: la vida se muestra con dos decimales (`%.2f`) y acompañada de una barra de almohadillas generada con `"#".repeat(...)`, de forma que el jugador ve de un vistazo cómo evoluciona el combate:

```
Tu vida es ### 36.59 y la del rival es ###### 60.00
Que quieres hacer
1. Atacar
2. Curarte
...
7. Cambiar pekemen
Elecion:
```

3.6. Flujo de un turno de combate

Cada vuelta del bucle de turnos sigue siempre el mismo orden, que es la base de las reglas del juego:

1. Se muestra el estado: barras y valores de vida de ambos bandos y el menú de acciones.
2. Se lee la opción del jugador y se ejecuta con un `switch` (daño, curación, efecto, cambio, etc.).
3. Se aplican los estados alterados sobre el rival: quemadura (−15), veneno (−25) o avance del contador de ahogamiento.
4. Si el rival sigue con vida, la máquina actúa: se cura, lanza un crítico, hace un ataque desesperado o ataca normalmente, según su vida y el azar.
5. Se comprueba si ha caído algún pekemen: se busca el siguiente con vida de ese bando o se marca el fin del combate.
6. Si el combate no ha terminado, se vuelve al primer paso.

La ventaja de tipos y el daño se calculan con condicionales y operadores, como en este fragmento:

```
if ((tipo_jugador == "fuego" && tipo_rival == "planta")
        || (tipo_jugador == "planta" && tipo_rival == "agua")
        || (tipo_jugador == "agua" && tipo_rival == "fuego")) {
    ventaja = 1.25;                     // el jugador tiene ventaja
} else if (/* combinaciones inversas */) {
    ventaja = 0.75;                     // el jugador tiene desventaja
} else if (tipo_jugador == tipo_rival || tipo_jugador == "normal" || tipo_rival == "normal") {
    ventaja = 1.0;
}
// ataque básico: potencia aleatoria entre 10 y (fuerza + 1) * 10
potencia = (ram.nextDouble(fuerza_maxima_jugador) + 1) * 10;
vida_rival -= potencia * ventaja;
```

3.7. Uso de arrays (explicación detallada)

El juego depende de los arrays: sin ellos no se podría guardar el catálogo de criaturas, los equipos ni los datos del pekemen salvaje. Se usan los siguientes:

Array	Declaración	Qué almacena
pekemens	String[19][8]	Catálogo fijo de las 19 criaturas del juego.
tus_pekemens	String[6][8]	Equipo del jugador (hasta 6 pekemens).
pekemens_rival	String[6][8]	Equipo del rival en cada combate (hasta 6 pekemens).
lider_agua / lider_fuego / lider_planta	String[3][8]	Equipo de cada líder del modo historia.
pekemen_salvaje	String[2]	Vector unidimensional con el nombre y la experiencia del salvaje encontrado.

Vector de cada pekemen. Cada *fila* de esas matrices es un vector unidimensional de 8 posiciones (`String[8]`) con los datos de una criatura en un orden fijo:

Índice	0	1	2	3	4	5	6	7
Dato	número	nombre	tipo	vida	nivel	daño máximo	experiencia actual	exp. que da al ser derrotado

Esto permite acceder al dato concreto con dos índices: `tus_pekemens[posicion][3]` es la vida del pekemen activo. Para evitar que dos equipos compartan la misma criatura, al copiar una fila del catálogo se usa `clone()`, de modo que cada equipo tiene su propio vector y modificar la vida de uno no altera el catálogo:

```
tus_pekemens[0] = pekemens[0].clone();     // el jugador recibe una copia de Peplet
```

Operaciones que se realizan sobre los arrays:

- **Consulta y actualización:** al empezar cada turno se leen del array el tipo, la vida y el daño máximo del pekemen activo; al cambiar de pekemen o caer en combate se escribe de vuelta la vida (`tus_pekemens[posicion][3] = ...`).
- **Recorrido y conteo/búsqueda:** para saber si el equipo sigue vivo se recorre con un `for` hasta encontrar algún pekemen con vida mayor que 0; el mismo patrón localiza al siguiente rival disponible.
- **Selección:** el cambio de pekemen recorre el equipo, descarta el activo y las posiciones vacías y ofrece el resto al jugador.
- **Actualización masiva:** al *matar* a un salvaje se recorre todo el equipo para sumar experiencia y subir de nivel si procede.

- **Inserción:** al *capturar*, se busca la primera posición libre (`== null`) del equipo y se guarda allí una copia de la criatura capturada.

```
for (int i = 0; i < tus_pekemens.length; i++) {
    if (tus_pekemens[i] != null && tus_pekemens[i][1] != null && i != posicion
            && Double.parseDouble(tus_pekemens[i][3]) > 0) {
        // este pekemen está disponible para el cambio
    }
}
```

Control de índices. Los recorridos usan `i < array.length`, nunca un número fijo. Antes de leer una posición del equipo se comprueba que no sea `null` (hueco vacío), y la condición se escribe de forma que, si falla, Java no evalúa el acceso al dato (evaluación en cortocircuito de `&&`). Los índices del catálogo que se usan para salvajes (9 a 12) y líderes están dentro de las 19 posiciones de `pekemens`.

Por qué un array. El equipo tiene un número máximo de integrantes conocido (6) y hay que acceder a ellos por posición, recorrerlos y modificarlos; un conjunto de variables sueltas habría sido inmanejable. Guardar todos los datos como texto en un solo vector simplifica copiar una criatura completa, aunque obliga a convertir tipos (ver 3.3) y es una decisión que se podría mejorar (apartado 5).

4. Pruebas realizadas

Las pruebas se han hecho ejecutando `App.java` con una versión de Java 21 y simulando la entrada de teclado. Como el juego usa números aleatorios, en las partidas completas el resultado exacto cambia de una ejecución a otra; lo que se comprueba es el comportamiento esperado, no cifras concretas.

N.º	Prueba	Entrada	Resultado esperado	Resultado obtenido	Estado
1	Salir desde el menú (normal)	3	Muestra «Adios» y el programa termina	Muestra «Adios» y termina	Correcta
2	Opción inválida en el menú principal	7, luego 3	Muestra «del 1 al 3» y vuelve a preguntar	Mensaje de error y se repite el menú	Correcta
3	Pekemen inicial inválido	2, 9, 1	Avisa de que se elija entre 1 y 3 y repite	Muestra «Escoge entre el 1 y el 3» y repite	Correcta
4	Opción inválida en combate (límite)	9	Mensaje de opción válida del 1 al 7	Muestra el mensaje, pero el rival ataca igualmente en ese turno	Mejorable
5	Curarse con poción	2	Vida +20 y poción gastada	«Has gastado tu ultima pocion»; vida 16,59 pasó a 36,59	Correcta
6	Curarse sin poción (límite)	2 (segunda vez)	Avisa de que no quedan pociones	«No te has podido curar porque no tienes pociones»	Correcta
7	Protegerse	5	El rival no hace daño normal	«Te has protegido»; la vida se mantiene en 18,97	Correcta
8	Efecto especial (agua contra planta)	6	Aplica el efecto de ahogar	«Le has ahogado»; el contador del efecto empieza	Correcta
9	Explotar	3	Rival a 0 de vida; victoria	«El rival ha sido derrotado», «Has ganado» y vuelta al menú	Correcta
10	Ganar atacando	1 repetido	La vida del rival baja hasta 0 y gana	Vida del rival 40, 33,25 y derrotado; «Has ganado»	Correcta
11	Perder el combate	2 repetido	Caen todos los pekemens del jugador; derrota	«Tu Peplet ha sido derrotado», «Has perdido el combate» y menú principal	Correcta

N.º	Prueba	Entrada	Resultado esperado	Resultado obtenido	Estado
12	Cambiar de pekemen sin otros disponibles (límite)	7	Avisa de que no hay otros con vida	«No tienes otros Pekemens disponibles con vida para cambiar.»	Correcta
13	Modo historia: tras ganar, menú del mundo	1, 1, 3	Tras la victoria aparece el menú líder / salvajes	Aparece el menú y el modo no termina	Correcta
14	Salvaje: capturar con hueco libre	2, 2, no	Intenta la captura en el primer hueco libre (60 %)	En 3 de 4 ejecuciones «Le has capturado»; en 1 «No le has capturado»	Correcta
15	Texto no numérico en un menú (error)	abc	Aviso y nueva petición	El programa se detiene con *NumberFormatException*	Fallo

Fragmento real de una ejecución (pruebas 4, 5 y 6): opción inválida, curación y curación sin poción.

```
Tu tipo es agua y el de tu rival es planta porque tiene a Terteg
Tu vida es #### 40.00 y la del rival es #### 40.00
Elecion: 9
Por favor, elige una opción válida del 1 al 7.
Tu vida es # 16.59 y la del rival es #### 40.00
Elecion: 2
Has gastado tu ultima pocion
Tu vida es ### 36.59 y la del rival es ###### 60.00
Elecion: 2
No te has podido curar porque no tienes pociones
Tu vida es # 18.97 y la del rival es ###### 60.00
```

Fragmento real del modo historia (prueba 13): tras ganar el primer combate aparece el menú del mundo y se encuentra un pekemen salvaje.

```
Elecion: 3
¡El Terteg del rival ha sido derrotado!
Has ganado
Que quieres hacer 1. luchar contra el lider o 2. luchar contra pekemens salvajes
2
Has salido: Bedef
Que quieres hacer matar(1) o capturar(2)
```

Conclusiones de las pruebas. Las acciones de combate, el menú, la condición de victoria y la de derrota funcionan como se diseñó. Las pruebas también sacaron a la luz dos limitaciones reales: (1) una entrada no numérica detiene el programa porque `Integer.parseInt` lanza una excepción que no se captura; y (2) una opción inválida en combate hace perder el turno. Ambas se recogen como mejoras para la segunda evaluación.

5. Dificultades, soluciones y ampliaciones

5.1. Dificultades encontradas y soluciones

- **Guardar y modificar los datos de cada criatura.** Al principio no estaba claro cómo representar varios datos de distinto tipo. Se resolvió con una matriz de texto en la que cada fila es una criatura, y conversiones con `parseInt` / `parseDouble` para operar.
- **Cambios que afectaban al catálogo.** Al asignar una fila directamente, los equipos compartían el mismo vector y bajar la vida de una criatura alteraba el catálogo. Se solucionó copiando con `clone()`.
- **Equipos incompletos.** Los huecos vacíos (`null`) provocaban errores al recorrer el equipo; se resolvió comprobando `!= null` antes de leer cada posición.
- **Controlar el final del combate.** Hubo que coordinar varios bucles anidados (menú, combate, turnos y mundo) con `break` y variables indicadoras para salir exactamente del bucle correcto.

- **Efectos con duración.** El ahogamiento necesita contar turnos; se implementó con un contador que actúa como estado (9 = inactivo, 0 a 3 = en curso).

5.2. Limitaciones conocidas

- Las entradas no numéricas detienen el programa (prueba 15).
- Una opción inválida de combate consume el turno (prueba 4).
- Se comparan textos con `==` en lugar de `equals`; funciona porque los literales comparten referencia, pero no es una práctica fiable.
- El cálculo de la ventaja de tipos está repetido varias veces en el código.

5.3. Ampliaciones propuestas para la segunda evaluación

- Validar toda la entrada con `try` para que un texto no numérico no cierre el programa.
- Agrupar el código repetido (ventaja de tipos, lectura de menús, cálculo de daño) en **métodos**.
- Sustituir las matrices de texto por **clases** (*Pekemen*, *Equipo*) con atributos tipados y usar `ArrayList`.
- Añadir una opción explícita para consultar el estado del equipo (vida, nivel y experiencia de cada pekemen).
- Guardar y cargar partidas en un fichero, y añadir más tipos, objetos y ataques distintos por criatura.
- Documentar y probar de forma sistemática con una tabla de casos y un registro de errores corregidos.